Variant Studio Internals

The short Variant Studio guide ran one prompt through two attempts and a reviewer, then kept the best tile. This notebook rebuilds that same run in minimal form — one attempt, one reviewer — and opens it up: the API objects Shepherd uses behind the recipe, and the flow trace it records along the way.

Primitives exposed here: a flow, retained runs, durable argument refs, artifact refs, a regenerated trace projection, and a task contract.

Setup

Load the launch helpers.

In [1]:
```
import pathlib
import sys


def _find_visual_artifact_example_root():
    cwd = pathlib.Path.cwd().resolve()
    candidates = []
    for base in (cwd, *cwd.parents):
        candidates.append(base)
        candidates.append(base / "examples" / "notebooks" / "visual_artifact
    for candidate in candidates:
        if (candidate / "shepherd_usecases" / "visual_artifact" / "launch.py
            return candidate
    raise RuntimeError(
        "Cannot find examples/notebooks/visual_artifact. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    )


example_root = _find_visual_artifact_example_root()
if str(example_root) not in sys.path:
    sys.path.insert(0, str(example_root))
try:
    from shepherd_usecases.visual_artifact import launch
    from shepherd_usecases.visual_artifact import viz
except Exception as exc:
    raise RuntimeError(
        "Could not import the visual-artifact notebook helpers. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    ) from exc

launch.bootstrap(example_root=example_root)
```

Build a run to inspect

`launch.open_workspace` opens the controlling scope — everything the run produces lives inside it. `launch.run_static` executes a `contour-map` attempt in that workspace and records its output. `launch.artifact_ref` mints a durable content-addressed handle to the artifact the attempt wrote.
`launch.run_with_artifact_ref` runs the reviewer in the same workspace, receiving that handle as the named argument `candidate` — which is what draws the dependency arrow from the attempt into the review in the trace viewer.

In [2]:
```
prompt = launch.default_prompt()
```

In [3]:
```
workspace = launch.open_workspace("visual-variant-studio-internals", prompt=
```

In [4]:
```
attempt = launch.run_static(
    workspace,
    name="contour-map",
    output_path=launch.ARTIFACT_PATH,
    output_text=launch.variant_html(prompt, "contour-map"),
    metadata={"variant": "contour-map"},
)
artifact_ref = launch.artifact_ref(attempt, label="contour-map")
reviewer = launch.run_with_artifact_ref(
    workspace,
    name="review",
    ref_name="candidate",
    artifact_ref=artifact_ref,
    output_path=launch.VERDICT_PATH,
    output_content=launch.review_content(prompt, {"contour-map": attempt}),
    after=[attempt],
)
```

That is the whole run. The rest of the notebook opens up what is behind it: the run record and argument refs, the regenerated flow trace, and the task contract.

Run record and argument refs

Every run Shepherd executes produces a **run record**: a content-addressed snapshot of the arguments it received. `launch.run_record` returns that record for any run in the workspace.

- `args_ref` identifies the argument bundle by content address in the flow.
- `args_digest` is the hash — two runs given identical arguments share one record.
- `input_refs` are the artifact refs passed in as named arguments: here, the one `artifact_ref` the reviewer received as `candidate`.

`launch.run_args` unpacks the raw argument map, exposing those `input_refs` as dereferenceable handles. Together `record` and `args` let any downstream run re-cite exactly what the reviewer saw.

In [5]:
```
record = launch.run_record(workspace, reviewer)
args = launch.run_args(workspace, reviewer)
{
    "run_ref": reviewer.run_ref,
    "args_ref": record.args_ref,
    "args_digest": record.args_digest,
    "input_ref_count": len(args["input_refs"]),
    "artifact_ref": artifact_ref.to_json(),
}
```

Out[5]:
```
{'run_ref': 'run-535a290c0711',
 'args_ref': 'run-535a290c0711:args:e309dae276b92983',
 'args_digest': 'sha256:e309dae276b9298332f3bfd876364f2763c0f7625a0d8ebbddd
7ff63491d0bb8',
 'input_ref_count': 1,
 'artifact_ref': {'kind': 'p030.run_artifact_input.v1',
  'run_ref': 'run-b5f6abd92144',
  'output_id': 'p030-run-output:1453ad85ea9dc852398a867b2e297277163306dc436
46b7442abdcc0e775d659',
  'output_name': 'workspace',
  'binding': 'workspace',
  'path': 'index.html',
  'label': 'contour-map',
  'content_digest': 'sha256:f3cf42e7b493d1dade7ae3892141c5fd8646e531e5c6a8e
303f9304cad07b680'}}
```

Flow trace

`workspace.flow.trace()` regenerates the full trace from the live flow object — not from a stored snapshot — so it always reflects the current workspace state. `viz.flow_trace` renders it as an interactive viewer: each run is a lane, the effects it performed line up inside it in order, and an arrow between lanes means one run drew on another.

Here the attempt lane and the reviewer lane are both visible, with a dependency arrow from the attempt into the review — because the reviewer received `artifact_ref` and read it. Click any node to open the effect it represents.

In [6]:
```
viz.show(viz.flow_trace(workspace.flow.trace(), height="620px"))
```

Events

kind	run/flow	label	payload
flow.opened	flow-10cefa99dbb3		name='visual-variant-studio-interna
flow.fork.requested	run-b5f6abd92144		name='contour-map', after=[]
run.lifecycle	run-b5f6abd92144		task_id='shepherd_usecases.visual_a task_version='v1', status='retained
provider.invocation	run-b5f6abd92144		execution_id='task-execution-245945 invocation_id='static:bf5e83c43885' status='started', source='task_exec evidence_role='provider_provenance' payload={'args_digest': 'sha256:1f5ce5df59a5b802457ccd79b1a 'artifact_path': 'index.html', 'tas 'shepherd_usecases.visual_artifact.
provider.invocation	run-b5f6abd92144		execution_id='task-execution-245945 invocation_id='static:bf5e83c43885' provider_event_kind='provider.invoc source='task_execution.metadata.pro event_id='static:bf5e83c43885:compl 'index.html', 'content_digest': 'sha256:83a73916eb76b536f48f34eade6 'launched_confined': False}
run.output.published	run-b5f6abd92144		output_name='workspace', output_id= output:1453ad85ea9dc852398a867b2e29 binding='workspace', output_world_o
flow.fork.requested	run-535a290c0711		name='review', after=['run-b5f6abd9
run.lifecycle	run-535a290c0711		task_id='shepherd_usecases.visual_a task_version='v1', status='retained
provider.invocation	run-535a290c0711		execution_id='task-execution-a07032 invocation_id='static:9df66d1a8f09' status='started', source='task_exec

Task contract

`tasks.static_artifact_task` is the underlying task function Shepherd called for the attempt run. `viz.task_contract` renders its contract: the typed inputs the task declares, the artifact path it writes to, and the docstring instruction the LLM receives when the task executes live.

The contract marks the boundary between the workspace-control layer and the generation step. Everything above the contract — the workspace, the run record, the artifact ref — is Shepherd's domain. Everything inside the task body is the generation step's domain.

In [7]:
```
from shepherd_usecases.visual_artifact import tasks
viz.task_contract(tasks.static_artifact_task)
```

Out[7]:
```
def static_artifact_task(
    repo: object,
    *,
    output_path: str,
    output_text: str | None = None,
    output_content: object | None = None,
    **artifact_refs: object,
) -> object:
    """Declare the static visual-artifact task executed by the provider runtime."""
    ...
```

Cleanup

`attempt.output().discard()` releases the attempt's retained output without selecting it. `reviewer.output().release()` releases the verdict. `workspace.close()` closes the controlling scope. The run ends, but its trace stays in the flow: the full record of effects each run performed and the dependencies between them.

In [8]:
```
attempt.output().discard()
reviewer.output().release()
workspace.close()
```